

Project 3: Enterprise RAG Chatbot

1. Introduction

The Enterprise RAG Chatbot is a document-driven intelligent system designed to provide accurate answers to user queries by combining retrieval mechanisms with large language models. Unlike basic chatbots, this system retrieves relevant information from an internal knowledge base and generates context-aware responses.

It is specifically structured for enterprise use cases such as policy lookup, internal documentation queries, and knowledge management systems.

2. Objective

The objectives of this project are:

- To build an enterprise-level knowledge assistant
- To enable querying over internal documents
- To improve response accuracy using retrieval-based context
- To maintain conversation flow using chat history
- To optimize performance using caching mechanisms

3. Technologies and Libraries Used

Programming Language: Python

Frontend: Streamlit

Embedding Model: SentenceTransformers

Vector Database: FAISS (local storage)

LLM Integration: Gemini API

Supporting Libraries

- dotenv
- NumPy
- pathlib
- custom utility modules

4. Architecture

System Workflow

1. System initializes the RAG pipeline
2. Documents are preloaded and indexed into FAISS
3. User enters a query
4. Query is converted into embeddings
5. Relevant documents are retrieved
6. LLM generates answer using retrieved context
7. Response is displayed and stored in chat history

Components

- **UI Layer:** Streamlit interface
- **Pipeline Layer:** Handles retrieval and generation
- **Vector Store:** FAISS index for similarity search
- **Memory Handling:** Chat history maintained in session
- **Caching Layer:** Prevents repeated API calls

5. Features

- Enterprise-level document querying
- Context-aware answer generation
- Chat history tracking
- Response caching for performance optimization
- Modular and scalable architecture
- Local vector storage using FAISS
- Clean and interactive user interface

6. Code Implementation

```
import streamlit as st

from app.main import setup_pipeline


st.set_page_config(page_title="Enterprise RAG Chatbot")
```

```
st.title("📚 Enterprise Knowledge Chatbot")
```

```
if "chain" not in st.session_state:
```

```
    st.session_state.chain = setup_pipeline()
```

```
if "chat_history" not in st.session_state:
```

```
    st.session_state.chat_history = []
```

```
# CACHE (prevents repeated API calls)
```

```
if "cache" not in st.session_state:
```

```
    st.session_state.cache = {}
```

```
query = st.text_input("Ask your question:")
```

```
if query:
```

```
    if query in st.session_state.cache:
```

```
        answer = st.session_state.cache[query]
```

```
    else:
```

```
        response = st.session_state.chain(query)
```

```
        answer = response["answer"]
```

```
        st.session_state.cache[query] = answer
```

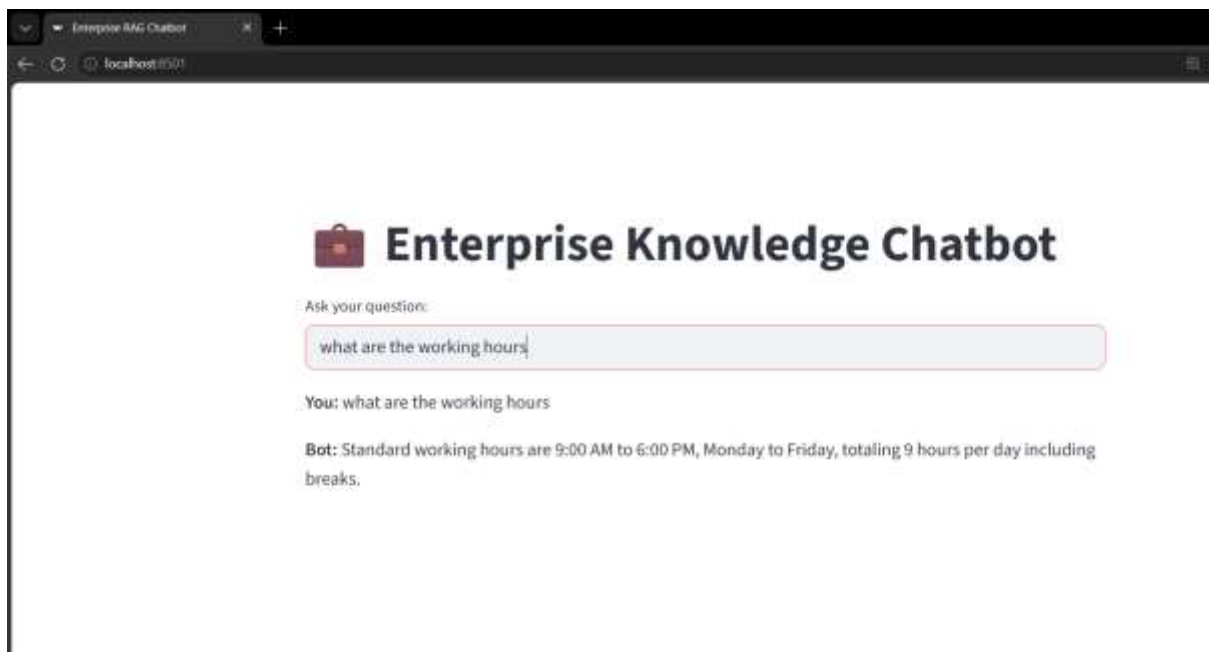
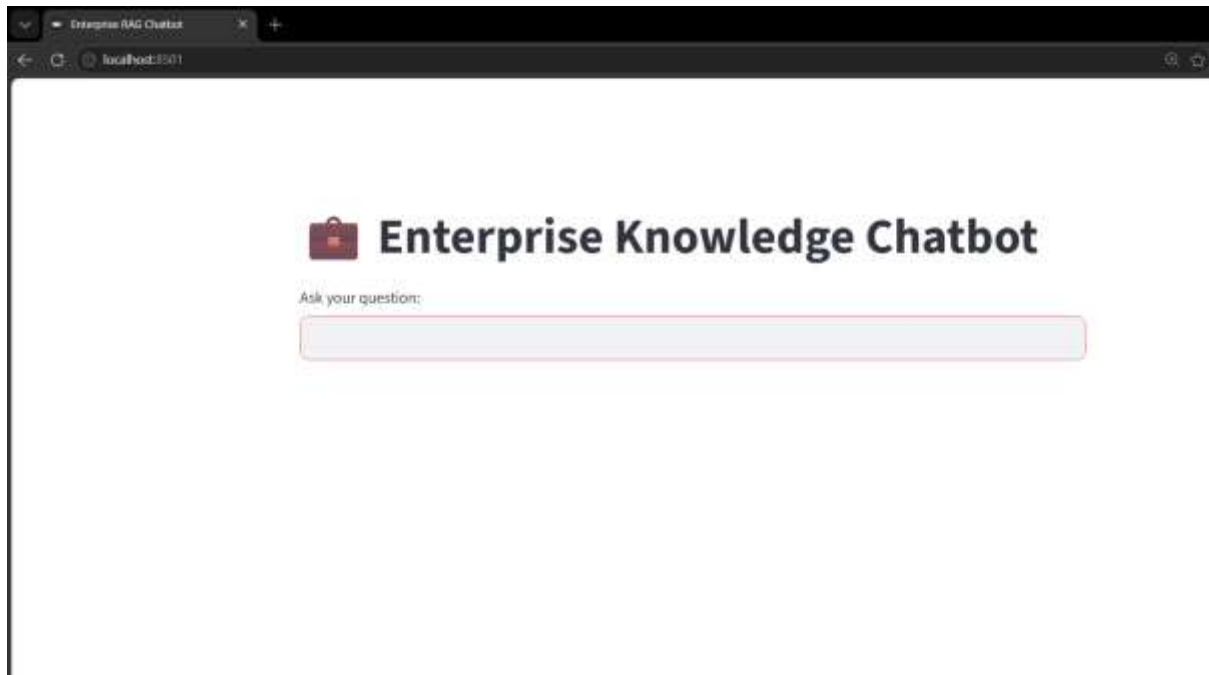
```
st.session_state.chat_history.append(("You", query))
```

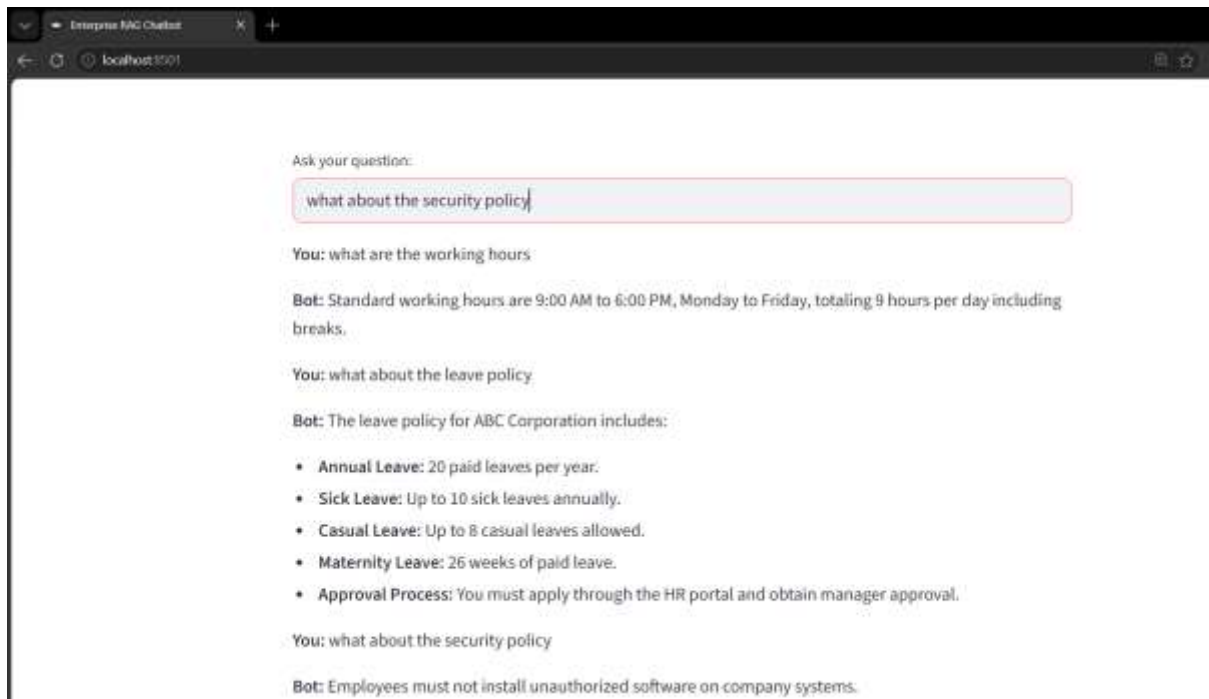
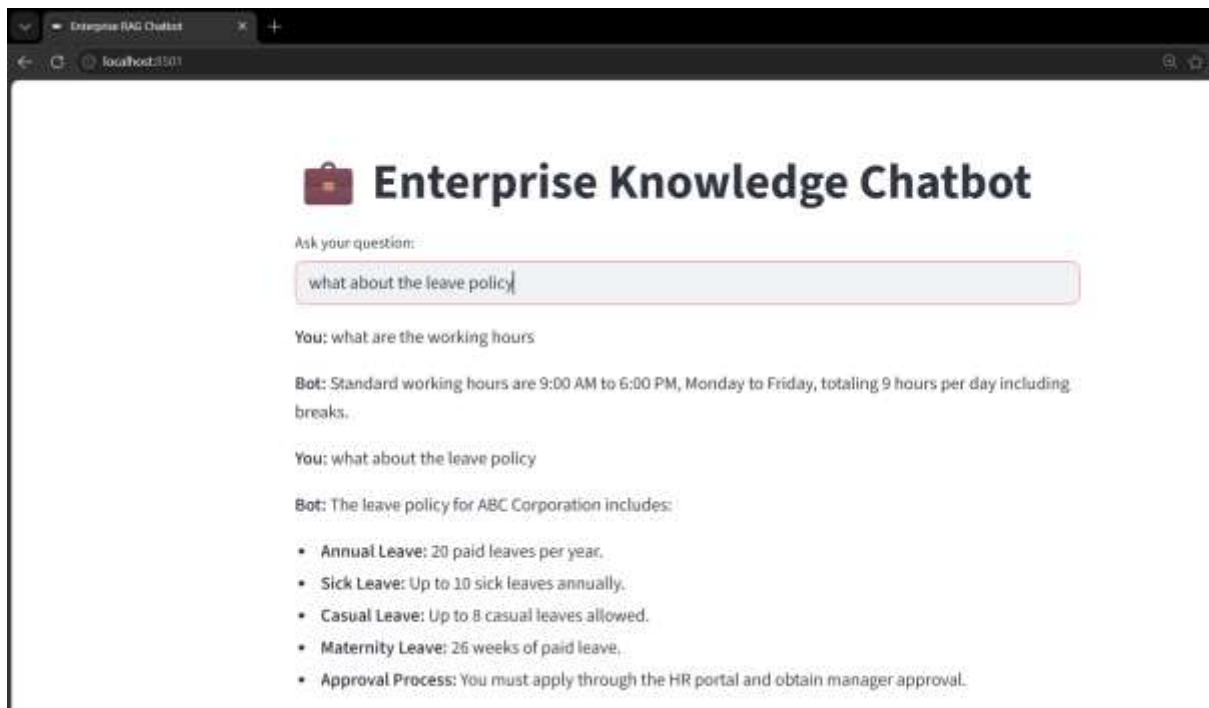
```
st.session_state.chat_history.append(("Bot", answer))
```

```
for role, msg in st.session_state.chat_history:
```

```
    st.write(f"**{role}**: ** {msg}")
```

7. Output





8. Conclusion

The Enterprise RAG Chatbot demonstrates how retrieval-augmented generation can be effectively used in real-world organizational settings. By combining document retrieval with LLM-based generation, the system provides accurate, efficient, and scalable solutions for enterprise knowledge management.

Project 4: Multi-Source RAG Chatbot

1. Introduction

The Multi-Source RAG Chatbot is an advanced document-based question answering system that integrates data from multiple sources such as files, websites, and databases. It enhances traditional RAG systems by allowing dynamic ingestion of diverse data formats and enabling efficient semantic search over a unified knowledge base.

This system is designed for real-world applications where enterprise data exists across multiple formats and locations.

2. Objective

The objectives of this project are:

- To build a unified knowledge retrieval system from multiple data sources
- To support querying across files, websites, and databases
- To improve response accuracy using retrieval-augmented generation
- To implement metadata-based filtering (e.g., department-wise search)
- To optimize storage using deduplication techniques

3. Technologies and Libraries Used

Programming Language: Python

Frontend: Streamlit

Embedding and Retrieval:

- LangChain
- SentenceTransformers

Vector Database: FAISS (local vector storage)

LLM Integration: Gemini API

Data Processing Libraries

- Pandas (CSV and database handling)
- PyPDF2 (PDF processing)
- python-docx (DOCX processing)

Web Scraping

- Requests
- BeautifulSoup

Database

- MySQL

Utilities

- hashlib (for deduplication)
- dotenv (environment variables)

4. Architecture

System Workflow

1. User provides input from multiple sources:
 - File upload (PDF, DOCX, CSV, TXT)
 - Website URL
 - MYSQL database
2. Data is extracted and converted into text
3. Documents are split into smaller chunks
4. Each chunk is converted into embeddings
5. Embeddings are stored in FAISS vector store
6. Duplicate content is removed using hashing
7. User enters a query
8. System retrieves relevant chunks using semantic search
9. Optional metadata filtering (department-based) is applied
10. LLM generates final response using retrieved context

Components

- Input Layer: Handles multi-source data ingestion
- Processing Layer: Cleans and splits data
- Storage Layer: FAISS vector database
- Retrieval Layer: Semantic search with filtering
- Generation Layer: LLM-based response generation

5. Features

- Multi-source data ingestion (files, web, database)
- Support for multiple file formats (PDF, DOCX, CSV, TXT)

- Website content extraction
- Database table ingestion (MySQL)
- Semantic search using embeddings
- Metadata-based filtering (department-wise retrieval)
- Deduplication using hashing to avoid redundant storage
- Persistent FAISS index storage
- Interactive chatbot interface
- Source tracking for retrieved answers

6. Code Implementation

```
import streamlit as st

from langchain_core.documents import Document

from langchain_text_splitters import RecursiveCharacterTextSplitter

import mysql.connector


from retriever import load_db


import pandas as pd

from docx import Document as DocxDocument

from PyPDF2 import PdfReader


import requests

from bs4 import BeautifulSoup

import hashlib


# Backend DB Config (NEW)

DB_CONFIG = {

    "host": "localhost",

    "user": "root",

    "password": "Kn@180597",

    "database": "rag_company"
```

```

}

# Load vectorstore
vectorstore = load_db()

# Splitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

st.set_page_config(
    page_title="Multi-Document RAG",
    page_icon="📄",
    layout="wide"
)

st.title("📄 Multi-Document RAG System")
st.markdown("Upload files, fetch web data, and query across multiple data sources using Retrieval-Augmented Generation.")

st.divider()

# ----- INPUTS -----
uploaded_files = st.file_uploader(
    "Upload files",
    type=["pdf", "docx", "csv", "txt"],
    accept_multiple_files=True
)

url = st.text_input("🌐 Enter website URL")

```

```
# SIMPLE TABLE SELECT (NO CONFIG IN UI)
```

```
table_name = st.selectbox(  
    " 📖 Select Database Table",  
    ["employees", "policies", "attendance"]  
)
```

```
# Button
```

```
process_btn = st.button("Process Data")
```

```
# ----- FILE READERS -----
```

```
def read_pdf(file):  
    reader = PdfReader(file)  
    return "".join([page.extract_text() or "" for page in reader.pages])
```

```
def read_docx(file):  
    doc = DocxDocument(file)  
    return "\n".join([p.text for p in doc.paragraphs])
```

```
def read_csv(file):  
    df = pd.read_csv(file)  
    return df.to_csv(index=False) # FIXED
```

```
def read_txt(file):  
    return file.read().decode("utf-8", errors="ignore")
```

```
def read_website(url):  
    try:  
        res = requests.get(url, timeout=10)  
        soup = BeautifulSoup(res.text, "html.parser")  
  
        for script in soup(["script", "style"]):
```

```

        script.extract()

    return soup.get_text(separator="\n")
except:
    return ""

def detect_department(filename):
    name = filename.lower()

    if "hr" in name:
        return "HR"
    elif "finance" in name:
        return "Finance"
    elif "it" in name:
        return "IT"
    else:
        return "General"

# MySQL Reader (UPDATED)
def read_mysql_database(table_name):
    try:
        conn = mysql.connector.connect(**DB_CONFIG)

        cursor = conn.cursor()
        cursor.execute(f"SELECT * FROM {table_name}")

        rows = cursor.fetchall()
        columns = [desc[0] for desc in cursor.description]

        df = pd.DataFrame(rows, columns=columns)

```

```
conn.close()
```

```
return df.to_csv(index=False)
```

```
except Exception as e:
```

```
    print("DB Error:", e)
```

```
    return ""
```

```
# ----- PROCESS -----
```

```
if process_btn:
```

```
    docs = []
```

```
# ♦ FILES
```

```
if uploaded_files:
```

```
    for file in uploaded_files:
```

```
        text = ""
```

```
        if file.name.endswith(".pdf"):
```

```
            text = read_pdf(file)
```

```
        elif file.name.endswith(".docx"):
```

```
            text = read_docx(file)
```

```
        elif file.name.endswith(".csv"):
```

```
            text = read_csv(file)
```

```
        elif file.name.endswith(".txt"):
```

```
            text = read_txt(file)
```

```
        if text.strip():
```

```
            docs.append(Document(
```

```
                page_content=text,
```

```
                metadata={
```

```
                    "source": file.name,
```

```
        "department": detect_department(file.name),
    }
))
```

♦ WEBSITE

if url:

```
    text = read_website(url)
```

if text.strip():

```
    docs.append(Document(
        page_content=text,
        metadata={"source": url, "type": "web"}
    ))
```

else:

```
    st.warning(" ⚠ Failed to extract website content")
```

♦ MYSQL DATABASE (BACKEND CONFIG)

if table_name:

```
    text = read_mysql_database(table_name)
```

if text.strip():

```
    docs.append(Document(
        page_content=text,
        metadata={"source": f"mysql:{table_name}", "type": "database"}
    ))
```

else:

```
    st.warning(" ⚠ Failed to read database")
```

----- DEDUP + INDEX -----

if docs:

```
    chunks = splitter.split_documents(docs)
```

```

existing_docs = vectorstore.similarity_search("", k=1000)

existing_hashes = {
    d.metadata.get("hash") for d in existing_docs if "hash" in d.metadata
}

new_chunks = []

for chunk in chunks:

    chunk_hash = hashlib.md5(chunk.page_content.encode()).hexdigest()

    if chunk_hash not in existing_hashes:
        chunk.metadata["hash"] = chunk_hash
        new_chunks.append(chunk)

if new_chunks:
    vectorstore.add_documents(new_chunks)
    vectorstore.save_local("vectorstore/faiss_index")

    st.success(f"✅ Added {len(new_chunks)} new unique chunks")
else:
    st.warning("⚠️ No new content (duplicates skipped)")

else:
    st.warning("⚠️ No valid data provided")

# ----- CHAT SECTION -----

st.title("💬 Ask Questions")

dept_filter = st.selectbox(
    "Select Department",

```

```
["All", "HR", "Finance", "IT", "General"]
)

query = st.text_input("Ask your question:")

if query:
    from retriever import retrieve
    from llm import ask_llm

    docs = retrieve(query, department=dept_filter)

    context = "\n\n".join([d.page_content[:300] for d in docs])

    answer = ask_llm(context, query)

    st.write("### 🤖 Answer")
    st.write(answer)

    with st.expander("📄 Sources"):
        for d in docs:
            st.write(f"- {d.metadata.get('source')} ({d.metadata.get('type')})")
```

7. Output

Enterprise RAG Chatbot

Multi Document RAG

localhost:5172

Deploy



Multi-Document RAG System

Upload files, fetch web data, and query across multiple data sources using Retrieval-Augmented Generation.

Upload files

Upload

200MB per file • PDF, DOCX, CSV, TXT

Enter website URL

Select Database Table

employees

Process Data

Select Database Table

employees

employees

policies

attendance

Select Department

All

Ask your question:

Deploy

Select Database Table

employees

Process Data

 **Ask Questions**

Select Department

All

Ask your question:

Enterprise HRG Chatbot Multi-Documen HRG to policy.pdf localhost:19002 Deploy

Select Database Table

admin/ruwec

All

HR

Finance

IT

General

All

Ask your question:

Integrate RAG Chatbot

Multi-Document RAG

localhost:3052

Deploy



Multi-Document RAG System

Upload files, fetch web data, and query across multiple data sources using Retrieval-Augmented Generation.

Upload files

 hr_employee.csv
133 KB

 hr_handbook.docx
15.2 KB

 hr_policy.pdf
207 KB

+

Enter website URL

https://en.wikipedia.org/wiki/Human_resource_management

Select Database Table

polices

▼

Process Data

Deploy

Process Data



Ask Questions

Select Department

HR

▼

Ask your question:

what is the policy of remote work?

 Answer

Employees can work remotely twice a week with manager approval.

>

Sources

8. Conclusion

The Multi-Source RAG Chatbot demonstrates a scalable and flexible approach to building intelligent knowledge systems. By integrating multiple data sources and applying efficient retrieval techniques, the system ensures accurate and context-aware responses.

This project can be extended further with advanced filtering, real-time data ingestion, and conversational memory to support enterprise-level deployments.